Linear-Time Permutation Check

Improved Prover, Verifier & Soundness
for SNARK Permutation Arguments

Benedikt Bünz, Jessica Chen, Zachary DeStefano — New York University

Why Permutation Checks Matter

Wiring check is fundamental to every SNARK:

$$f(x) = g(\sigma(x)) \quad \forall x \in B_\mu$$

Gate Check

Verify each gate computes correctly. Efficient via single sumcheck invocation — O(n) prover, O(log n) verifier.

Wiring Check (Bottleneck)

Prove input/output connections are consistent. Current methods commit n extra elements or use $O(\log^2 n)$-round GKR.

- HyperPlonk, Spartan, STARK: all rely on permutation checks
- Memory checking in zkVMs (read/write consistency)
- Lookup arguments for range proofs and non-arithmetic ops

Current Approaches & Their Limitations

Product Check (Plonk-style)

Commit to n extra field elements of full width.
Soundness error: $n/|F|$ (linear in input size).
For $n=2^{32}$, $|F|=2^{128}$: security only 2^{-96}.
Recent Fiat-Shamir attacks raise further concerns.

GKR-based (Spartan-style)

No extra commitments — $O(n)$ field ops.
But: $O(\log^2 n)$ rounds, proof size, verifier time.
Same poor soundness: $n/|F|$.
Higher prover latency, implementation complexity.

The Gap

Neither achieves the ideal: linear prover, log verifier, polylog soundness, no extra commitments, any PCS. Our BiPerm and MulPerm close this gap.

Key Idea: Sumcheck Formulation

Reducing Permutation to Sumcheck

Rewrite the permutation claim as a sum over an indicator function:

$$\sum_{x \in B_\mu} f(x) \cdot \tilde{1}_\sigma(x, \alpha) = g(\alpha)$$

where the indicator function is:

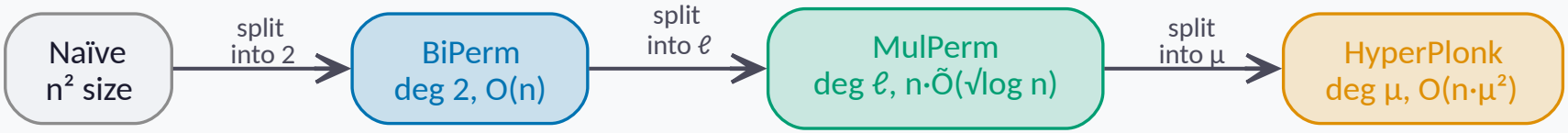
$$1_\sigma(X, Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \text{otherwise} \end{cases}$$

Key Question: How to Arithmetize the Indicator?

The efficiency of the entire protocol hinges on the degree of the arithmetization.
HyperPlonk: degree $\mu \rightarrow O(n \cdot \mu^2)$ prover time.
Our insight: decompose the indicator into fewer, larger parts → lower degree → faster prover.

Degree–Complexity Tradeoff

Splitting the indicator into fewer parts reduces prover cost:



$$\underbrace{\deg 1}_{n^2 \text{ size}} \longleftrightarrow \underbrace{\deg 2}_{\text{BiPerm}} \longleftrightarrow \underbrace{\deg \ell}_{\text{MulPerm}} \longleftrightarrow \underbrace{\deg \mu}_{\text{HyperPlonk}}$$

BiPerm (degree 2)

Split indicator into 2 halves.
O(n) field ops, O(μ/|F|) soundness.
Requires sparse PCS (Hyrax, Dory, KZH).

MulPerm (degree ℓ)

Split into ℓ = √μ parts.
n·Õ(√log n) field ops, any PCS.
Polylog soundness: O($\mu^{3?2}$/|F|).

BiPerm — Linear-Time Protocol

BiPerm: Splitting the Indicator

Factor the indicator into left and right halves of the output bits:

$$\mathbb{1}_{\sigma}(X, Y) = \mathbb{1}_{\sigma_L}(X, Y_L) \cdot \mathbb{1}_{\sigma_R}(X, Y_R)$$

This yields a degree-3 sumcheck:

$$\sum_{x \in B_\mu} f(x) \cdot \widetilde{\mathbb{1}}_{\sigma_L}(x, \alpha_L) \cdot \widetilde{\mathbb{1}}_{\sigma_R}(x, \alpha_R) = g(\alpha)$$

Advantages

Single degree-3 sumcheck — no grand product.
n-sparse polynomials of size $n^{1.5}$.
O(n) field ops on top of witness commitment.
O(μ) verifier, O(μ/|F|) soundness.

Limitation

The multilinear extensions have $n^{1.5} = n\sqrt{n}$ entries (n-sparse).
Requires PCS where commit/open scales with sparsity.
Compatible: Hyrax, Dory, KZH.
Not efficient with: KZG, FRI, Ligero.

BiPerm: Properties & PCS Compatibility

$$\text{Soundness:}\quad O\left(\frac{\log n}{|\mathbb{F}|}\right),\quad \text{Prover: } O(n),\quad \text{Verifier: } O(\log n)$$

PCS	Preprocess	Prove	Open	Compatible?
Hyrax/KZH	O(n)	O(n)	O(√n)	✓ Optimal
Dory	O(n)	O(n)	O(√n)	✓ Optimal
KZG	$O(n^{1.5})$	$O(n^{1.5})$	O(n)	✗ Superlinear
FRI/WHIR	$O(n^{1.5})$	$O(n^{1.5})$	O(n)	✗ Superlinear

Practical Takeaway

BiPerm is ideal when a sparse-friendly PCS is available. The prover cost is strictly O(n) field operations beyond the witness commitment. The preprocessed index is $O(n^{1.5})$ in size but only O(n) in weight (nonzero entries).

MulPerm — Universal Protocol

MulPerm: Multi-Part Decomposition

Generalize BiPerm: decompose indicator into ℓ parts, each mapping μ/ℓ bits:

$$\mathbb{I}_\sigma(X, Y) = \prod_{j=1}^{\ell} \mathbb{I}_j(X, Y^{(j)})$$

Each part is a product of eq polynomials:

$$p(x, \langle j \rangle) := \prod_{i=1}^{\mu/\ell} eq(\alpha(\langle j' + i \rangle), \tilde{\sigma}_{[\mu]}(\langle j' + i \rangle, x))$$

Two-Sumcheck Architecture

1st sumcheck: verify the degree-(ℓ+1) relation over the boolean hypercube.
2nd sumcheck: verify the partial product evaluations using preprocessed eq polynomials.
All polynomials are multilinear and of size O(n) — compatible with any PCS.

MulPerm: First Sumcheck

Prove the main permutation relation via a degree-(ℓ+1) sumcheck:

$$\sum_{x \in B_\mu} f(x) \cdot \prod_{j \in [\ell]} \tilde{p}(x, \langle j \rangle) = g(\alpha)$$

How It Works

Prover evaluates f and all partial products p over the boolean hypercube.
Round polynomial has degree ℓ+1 in the sumcheck variable.
After μ rounds, reduces to point evaluations of f and each partial product at a random challenge α.

Prover Cost Analysis

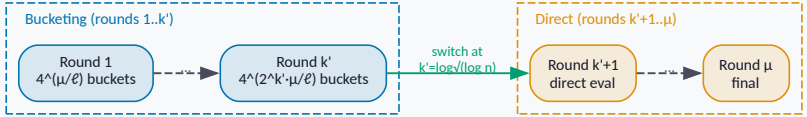
Naïve: O(n·ℓ) per round → O(n·ℓ·μ) total.
But partial products map {0,1} to {0,1}, so each can only take 4 possible functions.
This enables the bucketing algorithm for the second sumcheck.

Second Sumcheck & Bucketing Trick

$$\sum_{x \in \mathcal{B}_\mu,\, j \in [\ell]} \mathrm{eq}(\beta', x \| \langle j \rangle) \prod_{i=1}^{\mu/\ell} \mathrm{eq}\big(\alpha(\langle j' + i \rangle),\, \tilde{\sigma}_{[\mu]}(\langle j' + i \rangle, x)\big) = S_{\tilde{p}}$$

Bucketing Algorithm

Since eq maps to {0,1}, the round polynomial takes only finitely many distinct forms.
Group evaluations into buckets by their function signature.
Early rounds: few buckets → fast.
Late rounds: too many buckets → switch to direct evaluation.



Optimal Switch Point

Switch at round k' = log √(log n). Balances bucket count growth (doubly exponential) against direct cost reduction. Total: n·Õ(√log n).

MulPerm: Putting It Together

$$\underbrace{n \cdot \tilde{O}(\ell)}_{\text{1st sumcheck}} + \underbrace{n \cdot \tilde{O}(\mu/\ell) + \ell \cdot 2^{\ell}}_{\text{2nd sumcheck}} = n \cdot \tilde{O}(\ell + \mu/\ell) + \ell \cdot 2^{\ell}$$

$$n \cdot \tilde{O}(\sqrt{\log n})$$

field operations
(any PCS)

$$\ell = \sqrt{\mu} \implies n$$

$$\delta_s = \frac{O(\mu^{3/2})}{|F|} = \frac{\mathrm{polylog}(n)}{|F|}$$

Soundness error: polylogarithmic, not linear

MulPerm vs. Prior Work

vs. HyperPlonk sumcheck: $\mu^2 \to \sqrt{\mu}$ (prover speedup).
vs. GKR: $\log^2 n \to 2 \log n$ (rounds/verifier).
vs. ProdCheck: $n/|F| \to \mathrm{polylog}(n)/|F|$ (soundness).
No extra commitments beyond witness.

Applications & Extensions

Comparison of Permutation Check PIOPs

PIOP	|Comm.|	Rounds	P field ops	V time	Soundness
ProdCheck	$n+\log n$	$\log n$	n	$\log n$	$n/\|F\|$
GKR-based	$\log^2 n$	$\log^2 n$	n	$\log^2 n$	$n/\|F\|$
HyperPlonk	$\log^2 n$	$2 \log n$	$n \cdot \tilde{O}(\log n)$	$\log n$	$\log^2 n/\|F\|$
BiPerm	$\log n$	$\log n$	n	$\log n$	$\log n/\|F\|$
MulPerm	$\log^{1.5} n$	$2 \log n$	$n \cdot \tilde{O}(\sqrt{\log n})$	$2 \log n$	$\log^{1.5} n/\|F\|$

Key Advantages

BiPerm: only protocol with strictly linear prover and log soundness — but requires sparse PCS.
MulPerm: near-linear prover with any PCS. First to achieve polylog soundness without GKR.
Both eliminate the need for n extra commitments.

Applications: HyperPlonk & Spartan

Improving HyperPlonk

Replace permutation check with BiPerm/MulPerm.
Result: single witness commitment, no extra oracles.
Batch gate-check and perm-check sumchecks.
Especially valuable for sparse/low-weight witnesses.

Improving Spartan (SPARK)

SPARK encodes sparse matrices via row/col/val polynomials. Verifier delegates MLE evaluation to prover.
Replace GKR-based memory checking with our lookup argument.
Reduces soundness, verifier time, proof size.

SPARK sparse polynomial encoding:

$$(\mathrm{r}_x, r_y) = \sum_{j \in \mathcal{B}_\mu} \widetilde{\mathrm{val}}(j) \cdot \tilde{1}_{\mathrm{row}}(j, r_x) \cdot \tilde{1}_{\mathrm{col}}(j, r_y)$$

Structure matches our lookup argument — row, col are maps between boolean hypercubes

Generalization to Lookup Arguments

Replace injective permutation with arbitrary map to a table:

$$g(x) = \sum_{y \in B_\kappa} t(y) \cdot 1_\rho(x, y) \quad \forall x \in B_\mu$$

MulLookup

First lookup argument with polylog soundness error.
Prover: n·Õ(√log T) field ops for T < n.
Only commits to the map — no extra Theta(n) data.
Table dependency: single MLE evaluation.

Prover-Provided Maps

When map is not preprocessed, must prove well-formedness.
Binary check: each output bit is 0 or 1.
Permutation check: commit to inverse τ, verify σ(τ(x))=x.
Adds O(n log log n) overhead.

$$n \cdot \tilde{O}(\sqrt{\log T})$$

Prover cost for tables up to T < n^{log n}

Key Takeaways

BiPerm

1

Strictly linear prover time.
Single degree-3 sumcheck.
log n verifier, log n/|F| soundness.
Requires sparse-friendly PCS.

MulPerm

2

n·Õ(√log n) prover, any PCS.
Two sumcheck protocols.
Polylog soundness: $\mu^{3?2}/|F|$.
No extra commitments.

Lookups & Apps

3

First polylog-sound lookup.
Improves HyperPlonk, Spartan.
R1CS-style GKR for layered circuits.
Drop-in replacement.

$$n \frac{}{|\mathbb{F}|=2^{-96}} \longrightarrow \frac{\mathrm{polylog}(n)}{|\mathbb{F}|}=2^{-120}$$

Soundness improvement for $n = 2^{32}$, $|F| = 2^{128}$

References

Bünz, Chen, DeStefano — Linear-Time Permutation Check. 2025.

[CBBZ23] Chen, Bowe, Bünz, Zhao — HyperPlonk. EUROCRYPT 2023.

[Set20] Setty — Spartan: Efficient and General-Purpose zkSNARKs. CRYPTO 2020.

[STW24] Setty, Thaler, Wahby — Lasso: Lookup Arguments. EUROCRYPT 2024.

[ST25] Setty, Thaler — Shout: Efficient Range-Check Protocol. ePrint 2024.

[GKR08] Goldwasser, Kalai, Rothblum — Delegating Computation. STOC 2008.

Thank You

Questions?

Appendix / Backup Slides

Backup: Prover-Provided Permutation

Binary Map Check

Prove each output bit of σ is in {0,1}.
Use sumcheck to verify the constraint:

$$\sum_{x \in \mathcal{B}_\mu,\, i \in [\mu]} \mathrm{eq}\big((x, \langle i \rangle), s\big) \cdot \tilde{\sigma}_{[\mu]}(i, x)\big(1 - \tilde{\sigma}_{[\mu]}(i, x)\big) = 0$$

Inverse Trick

To prove σ is a permutation (bijective):
Commit to inverse τ.
Run permutation check: σ(τ(x)) = x.

Batching

Combine f=g and permutation into single sumcheck via random linear combination:

$$f'(y) = y + R \cdot f(y), g'(y) = \tau(y) + R \cdot g(y)$$

Backup: Soundness & Formal Statement

The eq polynomial (building block of all indicator functions):

$$eq(X, Y) = \prod_{i=1}^{\mu} (X_i Y_i + (1 - X_i)(1 - Y_i))$$

HyperPlonk arithmetization (degree-μ indicator):

$$\tilde{1}_o(X, Y) = \prod_{i=1}^{\mu} eq(\tilde{o}_i(X), Y_i)$$

Formal claim (Lemma 4):

$$\sum_{x \in B_\mu} f(x) \cdot \tilde{1}_o(x, y) = g(y) \ \forall y \in B_\mu$$

Soundness Bound

BiPerm: $O(\mu/|F|)$. MulPerm: $O(\mu^{3?2}/|F|)$.
Both polylogarithmic in n, compared to linear $n/|F|$ of product checks.
For $n=2^{32}$: improvement from 2^{-96} to 2^{-120}.

Backup: Full Degree–Complexity Landscape

$$\underbrace{\deg 1}_{n^2 \text{ size}} \longleftrightarrow \underbrace{\deg 2}_{\text{BiPerm}} \longleftrightarrow \underbrace{\deg \ell}_{\text{MulPerm}} \longleftrightarrow \underbrace{\deg \mu}_{\text{HyperPlonk}}$$

Protocol	Degree	Index Size	P field ops	Soundness
Naïve (full MLE)	1	n^2	n	log n/|F|
BiPerm	2	$n^{1.5}$ (sparse)	n	μ/|F|
MulPerm (ℓ=√μ)	√μ	n (dense)	n·Õ(√μ)	$\mu^{3?2}$/|F|
HyperPlonk	μ	n (dense)	n·μ^2	μ^2/|F|

Design Space Navigation

Moving left → right: degree increases, index size shrinks, prover cost grows.
BiPerm and MulPerm sit at the sweet spot: minimal extra commitments, near-optimal prover, and dramatically improved soundness over both endpoints.